

07 索引|

黃彬華編撰

- ✧ 索引概念
- ✧ MySQL 索引原理
- ✧ MySQL 建立索引優缺點
- ✧ 索引種類
- ✧ 索引資訊查詢
- ✧ 主鍵索引
- ✧ 唯一索引
- ✧ 一般索引
- ✧ 複合索引
- ✧ 回表與覆蓋索引查詢
- ✧ 索引效能評估
- ✧ 建立索引應注意事項

索引概念

黃彬華編撰

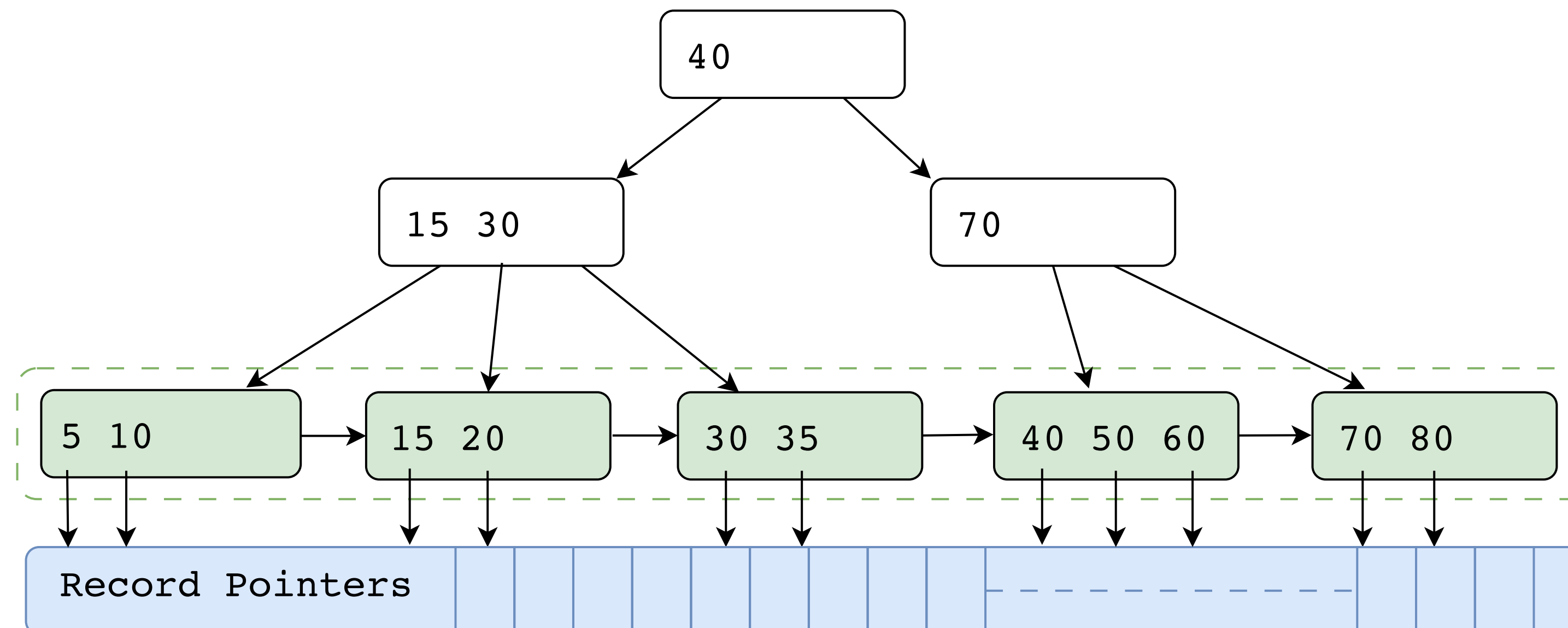
- ❖ 如果要找書的內容，一般會從目錄找起，然後再翻到指定頁面。如果沒有目錄，就必須從書的第一頁開始找起，曠日費時
- ❖ 資料庫的索引 (index) 類似書的目錄，沒有建立索引，就會從第一筆資料搜尋到最後一筆資料，執行效能會很差

MySQL 索引原理

黃彬華編撰

❖ MySQL 索引目前採 B+ Tree 方式

- 非葉子節點只存索引 (不存資料)，實際資料存在葉子節點 (末端點)，減少樹的高度，大大減少 I/O 存取次數。而且葉子節點之間用鏈結串起來，所以有以下優點：
 - ✦ 因為 B+ Tree 演算法不僅佳，而且可以減少 I/O 存取次數，所以資料搜尋速度快
 - ✦ 因為葉子節點之間使用鏈結串列，所以範圍查詢 (例如 WHERE id BETWEEN 15 AND 30) 佳



MySQL 建立索引優缺點

黃彬華編撰

❖ 優點

- 查詢速度快

❖ 缺點

- 資料異動 (INSERT、UPDATE、DELETE) 速度慢
 - ✦ 因為每次發生資料異動，索引就要更新一次
- 儲存索引，需佔用更多儲存空間

索引種類

黃彬華編撰

- ❖ 主鍵索引 (Primary Index)
- ❖ 唯一索引 (Unique Index)
- ❖ 一般索引 (Index)
- ❖ 複合索引 (Composite Index)

索引資訊查詢

黃彬華編撰

❖ 查詢表格的索引資訊

```
SELECT INDEX_NAME, COLUMN_NAME, INDEX_TYPE
```

```
FROM INFORMATION_SCHEMA.STATISTICS
```

```
WHERE TABLE_SCHEMA = 'db_name' AND TABLE_NAME = 'table_name';
```

- ✦ 雖然顯示的索引種類 (INDEX_TYPE) 為 BTREE，但實際上為 B+ Tree，可能因為名詞沿用，或是 "+" 與其他運算符號衝突

```
SHOW INDEX FROM table_name;
```

- ✦ SHOW INDEX 其實就是 SELECT STATISTICS 的結果，但無法指定欄位，而且某些欄位改用別名

範例

黃彬華編撰

❖ IndexInfoDemo

- ❖ 建立/移除方式

- 建立/移除 主鍵時就會自動 建立/移除 主鍵索引

- ❖ 特色

- 就是主鍵特色：欄位值不可重複、不可為 NULL

範例

黃彬華編撰

❖ PKIndexDemo

唯一索引

黃彬華編撰

- ❖ 建立/移除方式

- 建立/移除 唯一鍵時就會自動 建立/移除 唯一索引

- ❖ 特色

- 就是唯一鍵特色：欄位值不可重複、但值可以為 NULL

範例

黃彬華編撰

❖ UniqueIndexDemo

一般索引

黃彬華編撰

❖ 建立/移除方式

• 建立索引有下列方式

- ✦ `CREATE INDEX index_name ON table_name (column_name);`
- ✦ `ALTER TABLE table_name ADD INDEX index_name (column_name);`

• 移除索引

- ✦ `DROP INDEX index_name ON table_name`

❖ 特色

- 值可以重複，也可為 NULL
- 用在查詢條件欄位上，可以加快查詢速度

❖ 建立外鍵時沒有加上索引，MySQL 會自動幫外鍵建立一般索引來支援外鍵約束，確保父表在做 DELETE 或 UPDATE 時能快速查知是否已被 FK 參照

範例

黃彬華編撰

❖ IndexDemo

複合索引

黃彬華編撰

❖ 建立/移除方式

- 建立索引 (基本上與一般索引相同)
 - ✦ CREATE INDEX index_name ON table_name (column_list);
 - ✦ ALTER TABLE table_name ADD INDEX index_name (column_list);
- 移除索引
 - ✦ DROP INDEX index_name ON table_name

❖ 特色

- 將多個欄位一起加上索引，可以套用在前述任何類型索引上
- 查詢條件可能有多個欄位，如果都建立單一索引，MySQL 就會使用他認為最佳的索引來搜尋資料。如果查詢條件的多個欄位有優先序考量，可以建立複合索引，並將欄位優先序高低依照左到右排列

範例

黃彬華編撰

❖ CompositeIndexDemo

回表與覆蓋索引查詢 - 概論

黃彬華編撰

❖ 回表查詢 (back to table lookup)

- 因為次要索引 (非主鍵索引都是次要索引) 的葉節點只存次索引鍵與主鍵值，並不存整筆資料；如果還需要取出其他欄位值，就必須根據主鍵值再去主鍵索引取得完整資料，這個動作稱為回表查詢

❖ 覆蓋索引查詢 (covering index query)

- 如果查詢的所有欄位都在同一個索引內，不需再去主鍵索引查資料，就是覆蓋索引查詢 (不回表)
- 優點：
 - ✦ 不需要再次存取資料頁 → I/O 少
 - ✦ 查詢速度快 (特別在大量資料情況下差異明顯)

回表與覆蓋索引查詢 - 範例

黃彬華編撰

- ❖ 假設為 user 表格的 email 欄位建立 index

- create index idx_email on user(**email**);

- ❖ 回表查詢

- select id, **name**, email from user where email = 'a@example.com';
- 查詢的欄位 name 不在索引中，需要根據主鍵值 id 再去主鍵索引查出 name，屬於回表查詢
 - ✦ 想像成有個完整 user 主表儲存所有欄位，但 id 與 email 常用，所以建立一個快速表格只儲存 id 與 email 欄位方便查詢；但若要查詢 'a@example.com' 對應的 name，就必須先查詢 'a@example.com' 對應的 id，再由 id 回到主表去查詢 name

- ❖ 覆蓋索引查詢

- select id, name, email from user where email = 'a@example.com';
- 查詢的欄位 id, email 都在索引中，不需再去主鍵索引查資料，屬於覆蓋索引查詢

索引效能評估 - 1

黃彬華編撰

❖ 評估執行效能

- EXPLAIN + SELECT 語句
- 主要查看 type, key, rows, extra 等欄位資訊

索引效能評估 - 2

黃彬華編撰

- ❖ type：存取型態 (非常重要)
 - ALL：全表掃描 (最慢)
 - index：全索引掃描
 - range：索引範圍查詢 (例如：BETWEEN, >, <)
 - ref：使用非唯一索引做等值查詢
 - eq_ref：唯一索引做等值查詢 (例如主鍵 join)
 - const：單筆常數查詢 (最快)
 - system：表格只有一筆記錄，效能最佳

索引效能評估 - 3

黃彬華編撰

- ❖ key : MySQL 實際選用的索引 (如果是 NULL 表示沒用索引)
- ❖ rows : MySQL 預估需要查找的列數 (越小越好)
- ❖ Extra : 額外資訊，常見值：
 - Using where : 有套用 WHERE 條件
 - Using index : 覆蓋索引查詢 (不用回表，效率佳)
 - Using temporary : 建立臨時表 (通常出現在 GROUP BY, DISTINCT)
 - Using filesort : 需要額外排序 (ORDER BY 可能沒用到索引)
 - Using join buffer : 表示 join 沒用到索引，需要額外緩衝區

範例

黃彬華編撰

❖ IndexPerformanceDemo

建立索引應注意事項

黃彬華編撰

- ❖ 常使用在 WHERE、ORDER BY 或 GROUP BY 的欄位可以考慮建立索引，條件比對時執行效能會大幅提升
 - 如果該欄位值的變化小，例如只有 0 與 1，不需要建立索引，因為效果不顯著
- ❖ 索引的欄位儘量不為 NULL，以提升執行效能
- ❖ 如果將索引欄位放在函式內就會失去索引效果
- ❖ 建立索引會佔用空間，所以不要建立過多索引

練習 7-1

黃彬華編撰

- ❖ order_master 表格常用來查詢某客戶某段期間的訂單
 - 如何使用 EXPLAIN 語法評估執行效能
 - 應該如何建立索引來加快查詢速度
 - 注意：只需要評估 order_master 表格，不需要評估 order_detail 表格

- ❖ 建議：

- 先利用下列指令新增多筆資料，否則 EXPLAIN 的 rows 差異不大

```
insert into order_master(customer_id)
```

```
values
```

```
('C001'), ('C001'), ('C001'), ('C001'), ('C001'), ('C001'), ('C001'), ('C001'), ('C001'), ('C001'),  
('C002'), ('C002'), ('C002'), ('C002'), ('C002'), ('C002'), ('C002'), ('C002'), ('C002'), ('C002');
```